

SkillOpt: 自我进化代理技能的执行策略

杨一帆^{1,*‡} 龚子洋^{2,*} 黄伟权^{3,*} 杨启豪^{2,*} 周子威^{4,*} 黄子苏^{4,*} 李妍² 高雪梅¹ 戴琪¹ 刘蓓¹
邱凯¹ 杨雨晴¹ 陈东东¹ 杨雪^{2,‡} 罗冲¹ 微软² 上海交通大学³ 同济大学⁴ 复旦大学

现今的智能体技能要么是手工打造的，要么是一次性生成的，要么是通过松散控制的自我修订演化而来——这些方式无一能像深度学习优化器那样对技能进行优化，也无一能在反馈下稳定超越其初始水平。我们主张技能应当作为冻结智能体的外部状态进行 *trained* 优化，采用与权重空间优化相同的可复现规范。据我们所知，SkillOpt 是首个系统化的 *controllable* 文本空间技能优化器：通过独立优化模型将评分推演转化为对单一技能文档的有限增/删/改操作，且仅当编辑严格提升预留验证分数时才会采纳。文本学习率预算、拒批编辑缓冲及分阶段慢速/元更新机制，使得技能训练保持稳定的同时，在部署时不增加任何推理时模型调用。在六项基准测试、七种目标模型及三种执行框架（直接对话、Codex、Claude Code）中，SkillOpt 在全部 52 个（模型、基准、框架）评估单元上均取得最佳或并列最佳成绩，并击败了人类专家、一次性大语言模型、Trace2Skill、Text Grad、GEPA 和 EvoSkill 等所有单元竞争对手。在 GPT-5.5 上，它使平均无技能准确率在直接对话中提升 +23.5 个百分点，在 Codex 智能体循环内提升 +24.8 个百分点，在 Claude Code 环境下提升 +19.1 个百分点。迁移实验进一步表明，优化后的技能成果在跨模型规模迁移、Codex 与 Claude Code 执行环境间转移，以及未经优化直接应用于邻近数学基准测试时，仍能保持显著价值。

代码: <https://aka.ms/SkillOpt>

通讯作者: yifanyang@microsoft.com, yangxue2019-sjtu@sjtu.edu.cn

* *Equal contribution.* ‡ *Corresponding authors.*

1 引言

前沿语言模型正越来越多地被部署为代理，从单次提示调用者到配备工具、文件和验证器的多步骤执行框架[1-4]。在此类场景中，领域适应不再仅关乎模型权重或提示：它还需要改进代理收集证据、调用工具、遵循领域规范及格式化输出的{v*}方式[5,6]。代理技能为这种流程适应提供了自然接口[7,8]：技能是一种可移植的自然语言构件，封装了流程、领域启发式方法、工具策略、输出约束及故障模式，使冻结的代理能通过外部文本实现适应。

如果适配的重复对象是智能体的程序，那么技能文档本身应具备可训练性。然而，权重适配对于封闭前沿模型往往不可用且成本高昂

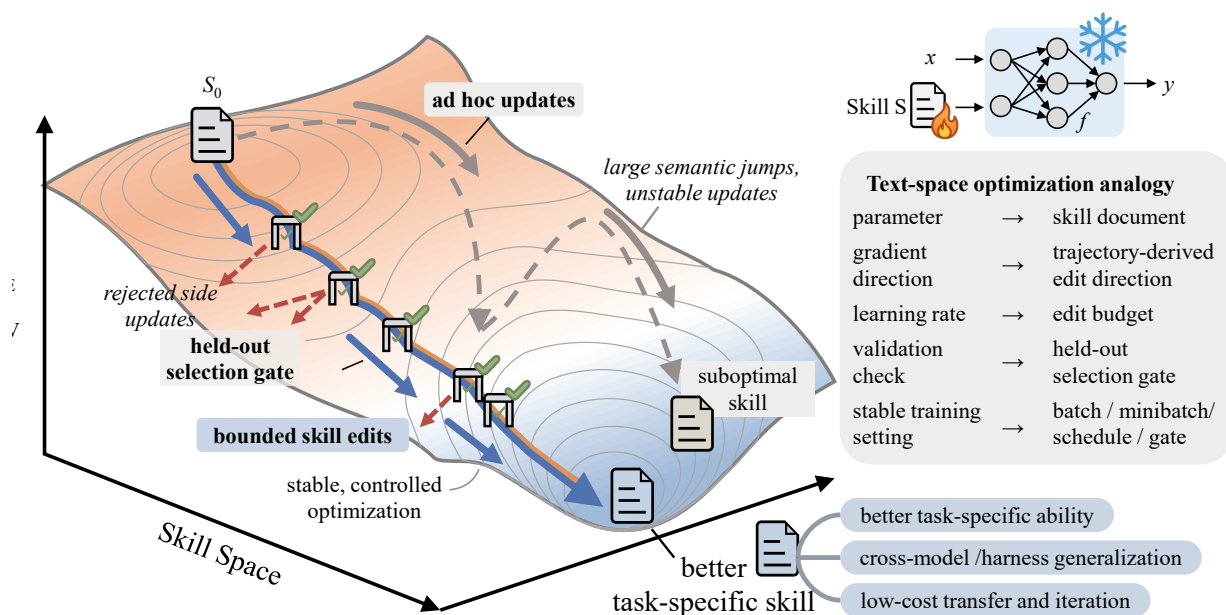


图1 SkillOpt概览。目标模型利用当前技能执行任务，额外的前沿优化器模型将轨迹转化为有界的添加/删除/替换技能编辑项，而预留的验证门仅接受能提升验证性能的编辑。被采纳的编辑将导出为可复用的技能构件，遭拒的编辑则转化为后续更新的负面反馈。

对于开放性问题，手动编写或一次性技能在目标领域或环境下显得脆弱。现有系统将执行经验转化为可复用的文本产物——提炼轨迹经验、通过失败分析优化技能文件夹、构建领域特定技能库，或根据轨迹反馈优化提示[9-13]——但一个更基本的问题仍未解决：若技能是适配层，应如何优化？我们的核心思想是将技能编辑视为可控的领域适配过程，以技能文档为外部状态，额外的前沿模型作为优化器，并对证据、步长、验证和更新方向实施训练式控制。

我们介绍了SkillOpt，一个面向智能体技能优化的文本空间优化器。给定目标领域、初始技能及待适配的模型，SkillOpt通过反复采样轨迹批次、分析成功与失败案例，并调用前沿优化模型提出结构化增删改操作建议。随后，它在文本学习率预算下汇总并排序候选编辑方案，对技能文档实施有界更新，并在采纳前通过预留的验证集评估候选技能。被拒绝的编辑将作为负反馈保留，而逐周期的慢速/元更新则维持更长周期的规律性。图1展示了这一循环的示意图。最终部署输出为一个约300至 $\{v^*\}$ 000令牌的紧凑best_skill.md文件，适配后的模型与执行框架保持固定不变。

深度学习的类比是操作性的，而非装饰性的。推出和反思批次大小控制着每次编辑所用证据的噪声；文本学习率和调度策略则决定了一个技能版本可以偏离前一个版本多远；保留的验证门起到了验证的作用；而逐周期的慢/元更新则像动量项一样，将稳定的编辑方向跨周期传递。这种稳定性至关重要：如果连续的技能修订移动过远或方向不一致，被拒绝的编辑和先前接受的编辑就不再提供有意义的优化历史。通过有界的、验证门控的更新，每次修订都足够接近前一次，使得后续优化器调用能够从哪些措施有效、哪些失败以及哪些应保留中学习。

据我们所知，我们首次对技能优化作为前沿智能体的领域适应训练方法进行了系统研究。我们在六个基准上评估了SkillOpt，涵盖问答、电子表格、文档处理、数学及具身决策等领域，测试了七个目标模型。

从前沿规模的GPT到小规模Qwen，在三种执行模式（直接对话、Codex框架、Claude Code框架）下进行评估。在52个（模型、基准、框架）评估单元中，SkillOpt在所有52项上均表现最佳或并列最佳。以GPT-5.5直接对话为例，它将SearchQA从77.7提升至87.3，SpreadsheetBench从41.8提升至80.7，OfficeQA从33.1提升至72.1，DocVQA从78.8提升至91.2，LiveMathematicianBench从37.6提升至66.9，ALFWorld从83.6提升至95.5（平均较无技能状态增益+23.5个点），同时以+5.4个点的平均优势超越由人工编写、单次LLM、Trace2Skill、TextGrad、GEPA及EvoSkill技能构成的最强*per-cell*基线。同一优化接口在Codex风格和Claude Code风格的执行循环中同样有效，使GPT-5.5分别较无技能状态提升+24.8和+19.1个点，并平均超越EvoSkill+14.0和+3.2个点。

学习到的成果还能迁移到精确训练设置之外。在GPT-5.4上训练的SpreadsheetBench技能，在我们测试的每一个较小GPT变体上都有提升；基于Codex训练的电子表格技能迁移到Claude Code时带来了+59.7分的增益；而OlympiadBench技能则在Omni-MATH[14]上产生了正向增益。这些迁移结果对论文的应用价值至关重要：一项技能只需优化一次，以文本形式审核后，便可在相关模型、工具或任务中重复使用，无需改变模型权重。我们的消融实验解释了其工作原理：有界文本学习优于不受控的重写，保留门控机制能防止有害建议累积，被拒步骤缓冲区将失败编辑转化为负面反馈，而逐轮慢速/元更新则在不膨胀部署技能的前提下改进了长周期优化。最后，各基准案例研究表明，习得技能始终保持紧凑（仅经1-4次采纳编辑后为300-2,000词元）、可检查且具有流程通用性而非特定于实例。

我们的贡献如下：

- 我们将智能体技能学习建模为对外部自然语言状态的优化过程，并提出了SkillOpt——一种与训练框架无关的优化器。该优化器具备以下特性：滚动批次处理、反思小批量更新、增删改编编辑操作、文本化学习率、调度机制、保留集验证、拒编辑缓冲池，以及分阶段慢速/元更新机制。公式标记{v*}保持原样。
- 我们通过六项基准测试、七个目标模型和三个执行框架展开了广泛的实证研究，结果表明SkillOpt在全部52个测试单元中表现最佳或并列最佳，并且在所有模型下均超越了无技能、人类技能、单次LLM技能、提示优化（TextGrad、GEPA）以及技能进化（Trace2Skill、EvoSkill）的基线方法。
- 我们通过组件消融和三种形式的迁移（跨模型、跨工具、跨基准）验证了优化设计，表明导出的技能构件紧凑、可重用，且无需更新模型权重即可部署。

2 相关工作

提示自动调优与智能体配置搜索。GEPA研究表明，轨迹反馈能引导提示的反思性进化，并在多项语言智能体任务上超越强化学习表现[13]。ABSTRAL与EvoTest将该理念从单一提示扩展到多智能体设计文档，并实现了无需梯度或微调的测试时智能系统进化[15,16]。这些方法将语言构件视为可优化对象，能直接利用执行反馈，但其主要优化目标仍是提示、系统设计或完整配置，而非可复用的领域适配。SkillOpt则创新性地优化了一种持久化技能文档——该文档可训练、验证、导出并与适配模型配合复用，从而将语言级可控性应用于稳定的程序化技能状态中。

技能构建与技能进化。SkillsBench与关于代理技能的SoK将技能视为可复用的程序性知识，涵盖工具策略、适用条件、执行

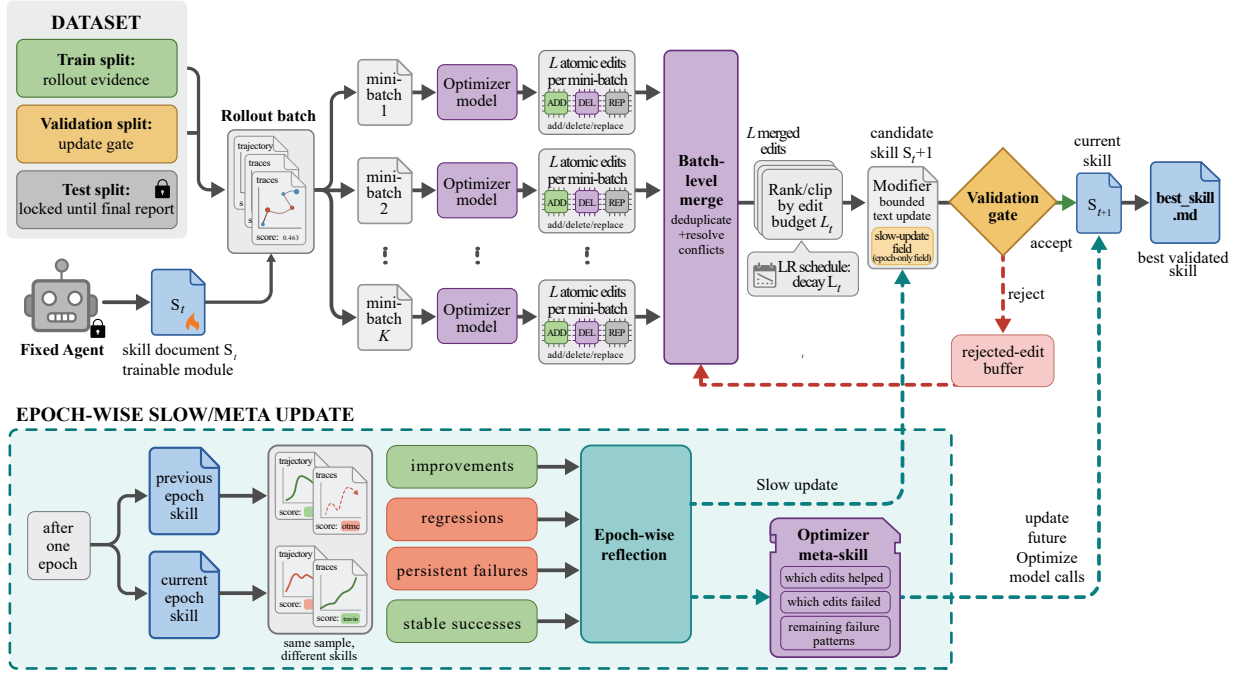


图2 SkillOpt流程。冻结的目标模型使用当前技能执行一轮批次；优化器模型对成功与失败进行小批量反思，提出有界的添加/删除/替换编辑建议，在预定的编辑预算下合并并排序这些编辑，且仅通过预留的验证关卡才接受候选技能。跨周期迭代中，慢速/元更新保留更长期的教训而不改变目标模型。

常规操作及支持资源[7, 8]。现有系统通过终身学习经验、轨迹课程、技能知识库或跨领域异构资源构建此类技能[9, 12, 17–19]，并通过故障分析、创建-评估-修订循环、协同进化的生成器与验证器、集体更新或强化学习进一步优化[10, 11, 20–26]。尽管这些研究侧重技能发现、知识库扩展、共享机制、进化搜索或策略优化，SkillOpt则聚焦更具体的问题：如何通过深度学习风格的控制手段（如轨迹批次、反思小批量、文本学习率、验证门限、拒绝编辑缓冲区及慢速/元更新）训练一个紧凑的领域技能。这形成了一套可控制、可审计的流程，在不改变模型权重的情况下生成可移植的 $\{v^*\}$ 。

3 方法

3.1 问题设置

一项技能 s 是在执行前插入到智能体上下文中的自然语言策略，这与近期将技能视为智能体可复用程序性知识的研究[7, 8]一致。在直接对话基准测试中，它被预置到系统或开发者指令前；在工具使用框架中，它则成为持久的程序性记忆。我们使用 M 表示通过技能优化进行行为适配的冻结目标模型。对于框架 h 、任务 x 和技能 s ，执行过程会产生轨迹 τ 和标量分数 r ：

$$(\tau(s), r(s)) = h(M, x, s), \quad r(s) \in [0, 1]. \quad (1)$$

给定训练、选择和测试分割 $D_{tr}, D_{sel}, D_{test}$ ，SkillOpt利用 D_{tr} 生成一组

候选技能 $\mathcal{C}(D_{\text{tr}})$ ，在 D_{sel} 上选择最佳技能，并在 D_{test} 上报告最终表现：

$$s_{\text{sel}}^* = \arg \max_{s \in \mathcal{C}(D_{\text{tr}})} \frac{1}{|D_{\text{sel}}|} \sum_{x \in D_{\text{sel}}} r(s), \quad (2)$$

$$\text{Test}(s_{\text{sel}}^*) = \frac{1}{|D_{\text{test}}|} \sum_{x \in D_{\text{test}}} r(s_{\text{sel}}^*). \quad (3)$$

训练集提供经验，选择集控制更新，测试集仅用于最终报告。优化器状态包含当前技能、最佳验证门控技能、缓存的技能哈希、一个周期内的拒绝步骤缓冲区，以及可选的慢速/元更新状态。只有被接受的最佳技能才会作为 $\{v^*\}$ 导出。

3.2 前向传递：展开证据

在每次优化步骤中，目标模型会利用当前技能从 D_{tr} 运行一个推出批次。该框架会记录任务元数据、消息、工具调用、观察结果、命令输出、最终答案、验证器反馈，以及特定于基准的上下文，如电子表格预览、文档引用或紧凑的执行轨迹。这个批次就是证据单元：小批次更新迅速但噪声较多，而较大批次则在技能变更前暴露出更多重复模式。该实现还支持累积机制，即多个推出批次被分别处理后合并为一次更新，从而将执行吞吐量与更新频率解耦。

3.3 反向传播：小批量反射

优化器模型将轨迹转化为技能编辑，遵循轨迹驱动的反思与提示演进的更广泛思路[13, 27, 28]。该模型首先将失败与成功案例分离，并将每组划分为反思小批次。这一步骤至关重要，因为单一轨迹往往只能产生零星修正，而小批次则能揭示可复用的程序性错误：智能体持续搜索错误来源、以错误格式书写答案或未能验证工具结果。失败小批次会提出缺失或纠正性规则；成功小批次则保留已有效的行为模式。每次反思会返回结构化的增/删/替换编辑项，或在重写模式下生成少量重写建议。

局部提案通过分层合并：首先分别整合失败驱动和成功驱动的编辑，然后优先考虑失败修正来合并它们。此步骤会过滤重复、矛盾及特定于示例的建议，之后优化器才选择最终的限定更新。

3.4 有界文本更新

SkillOpt 中的学习率类比是编辑预算 L_t ：即在步骤 t 应用的最大技能编辑次数。聚合后，优化器模型会根据预期效用对合并的编辑池进行排序，并截取前 L_t 个编辑。这是与临时提示重写的关键区别所在。无限制的改写可能会抹去有用规则、引入不兼容指令或导致局部过拟合；而有界更新则能在保持连续性的同时，仍允许技能获取新程序。SkillOpt 支持常量、线性、余弦及自主调度策略。默认的余弦调度会从较大的编辑开始，逐步衰减至较小的整合步骤。

所选编辑操作生成一个候选技能。在补丁模式下，编辑为局部操作，如追加、插入、替换和删除；而在重写模式下，选定建议会触发完整技能重写。步骤级编辑无法覆盖受保护的慢更新字段，从而确保快速局部变更与较慢的按周期整合保持分离。

3.5 验证门与拒编辑缓冲区

每个候选技能都在 D_{sel} 上使用相同的冻结目标模型和测试框架进行评估。若其表现优于当前选择分数，则成为新的当前技能；若同时超过迄今为止的最佳分数，则成为`best_skill.md`。否则将被拒绝。该机制将反思转化为提案-测试优化，而非无条件自我编辑，这一点至关重要——因为看似合理的文本诊断仍可能损害实际目标模型性能。

被拒绝的更新仍然有价值。优化器会记录一个包含观测到的失败模式的局部缓冲区，针对被拒绝的步骤，保存尝试过的编辑及其导致的分数下降。同一周期内后续的反思调用会接收这个缓冲区，因此优化器模型能避免重复失败的编辑，专注于未解决的失败。这为训练循环提供了负面反馈，而不会增加推理时的成本。

3.6 逐轮次慢速/元更新

快速更新从当前批次中学习；而基于周期的慢速/元更新则从相邻周期中汲取经验。在每个周期结束时，SkillOpt会选取上一周期技能与当前技能下相同的训练项进行采样，并将其归类为改进、退步、持续失败和稳定成功几类。优化模型会将简洁的纵向指导块写入受保护的慢更新字段，且该候选仍需通过验证门的检验。因此，慢速更新既能捕捉持久的领域经验，又能保持与步骤级编辑相同的安全检查机制。

元技能仅存在于优化器端。它总结了哪些编辑模式有效、哪些被拒绝，以及哪些失败在各轮迭代中持续存在。这份元指导会被前置到未来的优化器提示中，用于反思、合并和排序，但不会随目标模型一同发布。其优势在于关注点分离：部署后的技能保持紧凑且可移植，而训练过程则能从更丰富的编辑记录中受益。

3.7 与工具无关的部署

SkillOpt通过轻量级的适配器接口实现与训练框架无关，顺应了智能体嵌入工具使用和软件执行环境的广泛趋势[1, 2, 4]。适配器负责构建训练/评估批次、将当前技能注入智能体上下文、运行原生框架并返回评分后的轨迹。因此，同一优化器可适用于直接问答、电子表格执行、文档推理、多模态问答、实体环境以及Codex风格或Claude Code风格的执行循环。将技能视为适配层的主要实践优势在于：更强的优化器模型可离线训练出可复用的技能产物，生成的`best_skill.md`便能跨目标模型、训练框架及邻近基准进行部署或测试，而无需调整模型权重。

4 实验

我们评估了SkillOpt作为冻结代理的文本空间优化器：目标模型使用当前技能执行每项任务，同时离线优化器根据推演证据编辑该技能。实验回答了四个问题：(i) 优化后的技能是否优于无技能、人类技能、一次性LLM技能、提示优化（TextGrad、GEPA）以及技能进化（Trace2Skill、EvoSkill）基线？(ii) 相同的优化循环是否适用于直接对话、Codex和Claude Code框架，以及从尖端GPT到小型Qwen的七种目标模型？(iii) 哪些优化器控制因素起关键作用？(iv) 学习到的技能呈现何种形态，以及需要付出何种成本？

设定。我们在以下基准测试上报告了每个基准的原生硬分数或精确匹配准确率，这些测试包括SearchQA[29]、SpreadsheetBench[30]、OfficeQA[31]、DocVQA[32]、LiveMathematicianBench[33]（表格中简称为LiveMath）以及ALFWorld[34]，使用了两个模型家族：

Model	Skill source	SearchQA	Spreadsheet	OfficeQA	DocVQA	LiveMath	ALFWorld
No harness / direct chat							
GPT-5.5	No skill	77.7	41.8	33.1	78.8	37.6	83.6
	Human skill	81.8 _{+4.1}	72.9 _{+31.1}	66.9 _{+33.8}	90.1 _{+11.3}	38.4 _{+0.8}	91.8 _{+8.2}
	LLM skill	80.9 _{+3.2}	43.2 _{+1.4}	51.7 _{+18.6}	89.6 _{+10.8}	40.0 _{+2.4}	93.3 _{+9.7}
	Trace2Skill	82.4 _{+4.7}	49.6 _{+7.8}	65.7 _{+32.6}	90.6 _{+11.8}	52.0 _{+14.4}	87.3 _{+3.7}
	TextGrad	81.4 _{+3.7}	41.1 _{-0.7}	42.0 _{+8.9}	87.2 _{+8.4}	49.2 _{+11.6}	82.8 _{-0.8}
	GEPA	84.8 _{+7.1}	73.6 _{+31.8}	63.9 _{+30.8}	89.1 _{+10.3}	43.2 _{+5.6}	85.8 _{+2.2}
	SkillOpt	87.3_{+9.6}	80.7_{+38.9}	72.1_{+39.0}	91.2_{+12.4}	66.9_{+29.3}	95.5_{+11.9}
GPT-5.4	No skill	76.9	41.4	50.0	77.6	36.8	75.4
	Human skill	80.1 _{+3.2}	59.3 _{+17.9}	59.3 _{+9.3}	88.5 _{+10.9}	41.6 _{+4.8}	74.6 _{-0.8}
	LLM skill	79.4 _{+2.5}	46.1 _{+4.7}	20.4 _{-29.6}	90.4 _{+12.8}	36.8 _{+0.0}	83.6 _{+8.2}
	Trace2Skill	81.2 _{+4.3}	53.2 _{+11.8}	51.2 _{+1.2}	89.3 _{+11.7}	40.0 _{+3.2}	73.1 _{-2.3}
	TextGrad	82.5 _{+5.6}	38.6 _{-2.8}	54.7 _{+4.7}	85.3 _{+7.7}	36.6 _{-0.2}	78.4 _{+3.0}
	GEPA	82.4 _{+5.5}	61.1 _{+19.7}	60.4 _{+10.4}	88.3 _{+10.7}	41.6 _{+4.8}	74.6 _{-0.8}
	SkillOpt	83.1_{+6.2}	62.5_{+21.1}	62.8_{+12.8}	91.2_{+13.6}	44.0_{+7.2}	91.0_{+15.6}
GPT-5.4-mini	No skill	75.9	36.1	22.1	71.4	14.7	73.1
	Human skill	77.2 _{+1.3}	42.9 _{+6.8}	45.9 _{+23.8}	85.0 _{+13.6}	28.8 _{+14.1}	56.7 _{-16.4}
	LLM skill	76.1 _{+0.2}	36.8 _{+0.7}	36.6 _{+14.5}	86.4 _{+15.0}	28.0 _{+13.3}	65.7 _{-7.4}
	Trace2Skill	78.6 _{+2.7}	40.7 _{+4.6}	20.9 _{-1.2}	88.5 _{+17.1}	32.8 _{+18.1}	82.8 _{+9.7}
	TextGrad	77.5 _{+1.6}	38.2 _{+2.1}	30.0 _{+7.9}	84.0 _{+12.6}	27.2 _{+12.5}	70.9 _{-2.2}
	GEPA	79.4 _{+3.5}	42.5 _{+6.4}	45.3 _{+23.2}	83.7 _{+12.3}	27.2 _{+12.5}	81.3 _{+8.2}
	SkillOpt	80.2_{+4.3}	47.5_{+11.4}	48.8_{+26.7}	90.9_{+19.5}	32.8_{+18.1}	85.8_{+12.7}
GPT-5.4-nano	No skill	55.8	23.5	16.3	30.8	23.2	34.3
	Human skill	66.1 _{+10.3}	41.8 _{+18.3}	46.5 _{+30.2}	73.5 _{+42.7}	16.8 _{-6.4}	29.9 _{-4.4}
	LLM skill	62.4 _{+6.6}	33.6 _{+10.1}	12.8 _{-3.5}	71.7 _{+40.9}	20.0 _{-3.2}	65.7 _{+31.4}
	Trace2Skill	69.9 _{+14.1}	35.4 _{+11.9}	16.3 _{+0.0}	77.3 _{+46.5}	25.6 _{+2.4}	58.2 _{+23.9}
	TextGrad	73.4 _{+17.6}	32.5 _{+9.0}	38.7 _{+22.4}	68.3 _{+37.5}	20.8 _{-2.4}	42.5 _{+8.2}
	GEPA	73.2 _{+17.4}	37.2 _{+13.7}	45.0 _{+28.7}	71.2 _{+40.4}	24.8 _{+1.6}	68.7 _{+34.4}
	SkillOpt	74.8_{+19.0}	42.5_{+19.0}	50.0_{+33.7}	80.2_{+49.4}	27.2_{+4.0}	69.4_{+35.1}
GPT-5.2	No skill	71.9	38.2	34.9	73.1	20.8	68.7
	Human skill	75.8 _{+3.9}	48.2 _{+10.0}	55.2 _{+20.3}	89.0 _{+15.9}	23.2 _{+2.4}	56.7 _{-12.0}
	LLM skill	73.7 _{+1.8}	38.6 _{+0.4}	14.0 _{-20.9}	87.7 _{+14.6}	25.6 _{+4.8}	68.7 _{+0.0}
	Trace2Skill	79.2 _{+7.3}	43.9 _{+5.7}	43.0 _{+8.1}	87.7 _{+14.6}	28.8 _{+8.0}	76.9 _{+8.2}
	TextGrad	82.6 _{+10.7}	32.1 _{-6.1}	51.1 _{+16.2}	87.2 _{+14.1}	22.4 _{+1.6}	76.1 _{+7.4}
	GEPA	82.7 _{+10.8}	54.3 _{+16.1}	53.4 _{+18.5}	89.5 _{+16.4}	28.8 _{+8.0}	82.8 _{+14.1}
	SkillOpt	83.1_{+11.2}	57.1_{+18.9}	56.4_{+21.5}	89.6_{+16.5}	36.0_{+15.2}	85.1_{+16.4}
Qwen3.5-4B	No skill	68.1	9.3	14.5	86.9	22.4	30.6
	Human skill	66.3 _{-1.8}	16.4 _{+7.1}	22.7 _{+8.2}	87.8 _{+0.9}	18.4 _{-4.0}	28.4 _{-2.2}
	LLM skill	65.0 _{-3.1}	10.4 _{+1.1}	20.9 _{+6.4}	88.0 _{+1.1}	28.8 _{+6.4}	55.2 _{+24.6}
	Trace2Skill	68.5 _{+0.4}	19.3 _{+10.0}	16.3 _{+1.8}	88.0 _{+1.1}	27.2 _{+4.8}	64.9 _{+34.3}
	TextGrad	60.7 _{-7.4}	13.9 _{+4.6}	20.9 _{+6.4}	85.6 _{-1.3}	10.6 _{-11.8}	53.7 _{+23.1}
	GEPA	68.6 _{+0.5}	16.9 _{+7.6}	22.7 _{+8.2}	85.1 _{-1.8}	28.8 _{+6.4}	60.4 _{+29.8}
	SkillOpt	71.2_{+3.1}	23.9_{+14.6}	29.7_{+15.2}	89.0_{+2.1}	52.0_{+29.6}	81.3_{+50.7}
Qwen3.6-35B-A3B	No skill	72.7	38.2	45.9	87.6	31.2	59.7
	Human skill	74.1 _{+1.4}	44.3 _{+6.1}	41.9 _{-4.0}	88.4 _{+0.8}	29.6 _{-1.6}	44.8 _{-14.9}
	LLM skill	72.6 _{-0.1}	42.9 _{+4.7}	46.5 _{+0.6}	88.4 _{+0.8}	24.8 _{-6.4}	60.4 _{+0.7}
	Trace2Skill	75.4 _{+2.7}	33.2 _{-5.0}	32.0 _{-13.9}	90.4 _{+2.8}	29.6 _{-1.6}	70.9 _{+11.2}
	TextGrad	76.4 _{+3.7}	22.9 _{-15.3}	33.7 _{-12.2}	84.5 _{-3.1}	7.2 _{-24.0}	67.9 _{+8.2}
	GEPA	75.8 _{+3.1}	45.4 _{+7.2}	43.6 _{-2.3}	88.0 _{+0.4}	31.2 _{+0.0}	73.9 _{+14.2}
	SkillOpt	80.3_{+7.6}	47.5_{+9.3}	47.1_{+1.2}	91.4_{+3.8}	41.6_{+10.4}	82.1_{+22.4}
Codex harness							
GPT-5.5	No skill	81.8	27.5	38.3	87.2	35.2	—
	Human skill	84.1 _{+2.3}	50.7 _{+23.2}	40.0 _{+1.7}	88.8 _{+1.6}	48.8 _{+13.6}	—
	LLM skill	83.4 _{+1.6}	25.0 _{-2.5}	34.3 _{-4.0}	89.8 _{+2.6}	45.6 _{+10.4}	—
	EvoSkill	61.4 _{-20.4}	67.5 _{+40.0}	42.4 _{+4.1}	89.3 _{+2.1}	63.2 _{+28.0}	—
	SkillOpt	87.3_{+5.5}	85.0_{+57.5}	51.1_{+12.8}	92.2_{+5.0}	78.4_{+43.2}	—
Claude Code harness							
GPT-5.5	No skill	81.9	22.1	57.6	86.6	40.8	—
	Human skill	83.7 _{+1.8}	37.1 _{+15.0}	66.3 _{+8.7}	88.0 _{+1.4}	44.0 _{+3.2}	—
	LLM skill	82.4 _{+0.5}	37.1 _{+15.0}	56.4 _{-1.2}	89.6 _{+3.0}	44.8 _{+4.0}	—
	EvoSkill	84.0 _{+2.1}	75.0 _{+52.9}	70.3 _{+12.7}	87.2 _{+0.6}	52.0 _{+11.2}	—
	SkillOpt	85.9_{+4.0}	80.4_{+58.3}	71.5_{+13.9}	90.1_{+3.5}	56.5_{+15.7}	—

表1 保留测试集上的主要结果。得分以百分比表示；在每个模型-测试框架组合中，粗体标出各基准测试中的最佳表现条目，下划线标出次佳表现条目。蓝色单元格代表SkillOpt，绿色/红色小下标显示相对于同一模型在同一测试框架中 *No skill* 行的绝对变化量。我们未在Codex和Claude Code测试框架下列出ALFWorld数据，因为ALFWorld需要持续的具身环境交互。SkillOpt在表中所有测量单元格均取得最佳或并列最佳表现，相较无技能基线全程保持正向增益。

(a) Training set size				(b) Mini-batchsize				(c) Batchsize			
Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath
1 example	81.0	47.5	59.1	1	85.9	75.4	60.5	8	85.1	76.8	58.1
20% train	84.1	69.0	65.9	2	86.3	77.1	54.8	24	86.4	77.1	62.9
40% train	86.1	73.5	64.8	4	86.9	75.4	64.5	40	87.1	77.5	61.3
80% train	86.1	77.6	67.0	8	87.1	77.5	61.3	56	86.5	76.8	56.5
100% train	84.1	78.0	70.5	16	87.0	77.9	61.3	full epoch	87.2	75.0	53.2
				32	86.9	77.5	58.9				
(d) Learning rate				(e) Learning-rate scheduler				(f) Slow-update samples			
Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath	Setting	SearchQA	Spreadsheet	LiveMath
lr=1	85.5	77.5	62.1	constant	87.3	80.7	62.1	5	86.8	76.4	64.5
lr=2	86.7	77.5	60.5	cosine	87.1	77.5	61.3	10	86.4	74.3	65.3
lr=4	86.5	78.2	56.5	linear	87.2	72.9	62.9	20	87.1	77.5	61.3
lr=8	87.0	73.6	66.9					40	86.9	75.4	54.8
lr=16	86.8	78.2	65.3								

表2 文本优化器的超参数分析。每个面板从默认设置中更改一个标量或调度因子，除非另有说明。面板(a)将数据分割固定为4:1:5的训练/选择/测试比例；1样本、20%、40%和80%行使用训练分区的子集，100%行则复用了已完成4:1:5分割比例的运行结果。面板(b)扫描了反射小批量大小 B_m ；面板(c)扫描了推出批量大小 B 。

Component	Setting	SearchQA	SpreadsheetBench	LiveMath
Learning-rate form	lr=4 (default)	87.1	77.5	61.3
	dynamic lr	85.8	71.8	54.0
	without lr	84.6	75.7	57.3
Rejected buffer	with rejected buffer	87.1	77.5	61.3
	without rejected buffer	85.5	72.9	58.9
Slow/meta update	meta skill and slow update	87.1	77.5	61.3
	without meta skill	85.1	75.7	58.1
	without meta skill and slow update	86.3	55.0	59.7

表3展示了学习率形式、拒绝缓冲区和逐轮次慢/元更新的组件消融研究。浅蓝色行标记了每个组件组内的默认设置；学习率组采用了默认的lr=4设置。加粗数值标示了该组及基准测试中的最佳测量结果。无拒绝缓冲区行使用了匹配的无缓冲区消融设置。

GPT [35]与Qwen [36, 37]。该基准测试集刻意追求多样性——涵盖单轮问答（SearchQA、DocVQ A、LiveMathematicianBench MCQ）、多达24次工具调用的多轮工具循环（OfficeQA）、长达30轮交互且具备真实openpyxl/pandas运行环境的多轮代码生成（SpreadsheetBench，默认mode=multi），以及每幕多达50步的持久具身交互（ALFWorld）。基于数据集的运行采用源自相同数据集种子（split_seed = 42）的确定性训练/选择/测试划分；选择划分用于only以采纳或拒绝候选技能修改，所有报告分数均在互斥的保留测试划分上计算。因此报告数值衡量的是泛化能力，而非验证集拟合度。

默认优化器超参数。除非另有说明，SkillOpt使用四个训练周期，每步40个样本的推出批次大小，8的反思小批次大小（并行运行反思的16个分析工作线程，合并批次大小为8），文本学习率 $L_t = 4$ 带余弦衰减（下限 $L_t = 2$ ，可配置计划：常数、线性、余弦、自主），保留验证门控（严格大于当前选择分数——平局被拒绝），缓慢更新每周期20个抽样任务比较前一周期和当前周期技能，优化器侧的元技能将接受/拒绝模式总结为仅教师指导，patch编辑模式（替代方案为rewrite_from_suggestions），以及可选的最近失败提案拒绝编辑缓冲区。教师反思允许每小批次最多三轮细化。教师和学生调用默认使用medium推理努力。对于训练池严格受限的基准（LiveMathematicianBench：每周期35个训练项，推出批次200；ALFWorld：39个训练任务，140个选择和134个测试环境），每个基准配置相应缩放批次大小，同时保持相同的门控、调度器和缓慢/元更新机制。其他基准、基线和优化器协议细节见附录C。

(a) Cross-model transfer					
Source model	Target model	Benchmark	Baseline	Direct	Transferred
GPT-5.4	GPT-5.4	SpreadsheetBench	41.4	62.5	—
	GPT-5.4-mini		36.1	47.5	45.5 _{+9.4}
	GPT-5.4-nano		23.5	42.5	26.5 _{+3.0}
GPT-5.4	GPT-5.4	LiveMath	36.8	44.0	—
	GPT-5.4-mini		14.7	32.8	19.2 _{+4.5}
	GPT-5.4-nano		23.2	27.2	28.8 _{+5.6}
(b) Cross-harness transfer					
Source harness	Target harness	Benchmark	Baseline	Direct	Transferred
Codex	Claude Code	LiveMath	40.8	56.5	42.4 _{+1.6}
Claude Code	Codex		35.2	78.4	48.0 _{+12.8}
Codex	Claude Code	SpreadsheetBench	22.1	80.4	81.8 _{+59.7}
Claude Code	Codex		27.5	85.0	71.1 _{+43.6}
(c) Cross-benchmark transfer					
Source benchmark	Target benchmark	Model	Baseline	Direct	Transferred
OlympiadBench	Omni-MATH	GPT-5.4	56.6	—	60.3 _{+3.7}
		GPT-5.4-mini	34.8	—	36.6 _{+1.8}
		GPT-5.4-nano	38.8	—	40.1 _{+1.3}

表4 优化技能在三个轴向上的迁移情况。(a) *Cross-model*: 为源模型优化的技能部署在目标模型上。(b) *Cross-harness*: 在源框架内训练的技能在目标框架内评估, 全部基于GPT-5.5。(c) *Cross-benchmark*: 源基准技能在三个目标模型的目标基准上评估。基线是目标的无技能得分, 直接是领域内SkillOpt得分, 迁移则是直接应用源技能而不进行进一步优化。下标显示相对于目标基线的变化。GPT-5.4→GPT-5.4在(a)中的迁移单元格标记为—, 因为源与目标匹配(即未发生迁移); 我们仍在该行中提供GPT-5.4基线和领域内SkillOpt得分(取自表1)以供参考。(a)–(c)中其余每一行均为正向迁移(无任何行低于目标的无技能基线)。

背板系统。直接聊天模式通过在系统提示前添加技能描述, 通过单次聊天完成调用目标模型。Codex背板通过codex CLI在工作区写入沙箱中驱动目标模型[38]; SkillOpt将当前技能渲染为每任务专用的SKILL.md, 与任务文件并存, 并读取回一个精简的执行跟踪记录(`codex_trace_summary.txt`), 该记录会被纳入教师反思上下文, 因此优化器能从 *what the agent actually did* 中学习, 而不仅依赖最终答案。Claude Code背板通过claude CLI[39]实现了相同的工作区契约。这三种模式都使用相同的`best_skill.md`文件格式, 这正是第4.3节跨背板迁移实验得以实现的基础。

基线方法。我们对比了涵盖无适应、手工编写、一次性学习和学习型家族的七种基线方法: *no skill* (使用基准默认系统提示运行的冻结目标模型), *human skill* (每个基准精心编写的专家技能文档), *one-shot LLMskill* (由GPT-5.5根据高级任务描述生成且从未更新的单一技能), *Trace2Skill*[9] (轨迹级技能蒸馏), *TextGrad*[40] (梯度式自然语言提示优化), *GEPA*[13] (帕累托反射提示进化), 以及工具侧竞争者 *EvoSkill*[10] (失败分析下的技能文件夹进化)。所有基线方法均采用相同的目标模型、相同的保留测试集划分和相同的评分器, 确保比较仅聚焦于适应过程的选择, 而非提示模板或评分流程等次要因素。

4.1 主要结果

表1是主要结果矩阵。将每个(目标模型、基准测试、工具链)单元格视为一次比较, 并在无技能、人类技能、LLM技能、Trace2Skill、TextGrad等选项中选出最强表现,

在作为每单元竞争的GEPA和EvoSkill基线对比中，SkillOpt在52个评估单元中赢得或匹配了最佳测量结果。这种优势在不同模型规模间表现一致：SkillOpt在GPT-5.5、GPT-5.4、GPT-5.4-mini、GPT-5.4-nano、GPT-5.2、Qwen3.5-4B和Qwen3.6-35B-A3B的直接聊天测试中每个基准上都表现最佳，同时在Codex和Claude Code测试框架下的GPT-5.5模型上也保持领先。

对于不更新权重的方法而言，其性能提升幅度也异常显著。在GPT-5.5直接对话场景下，六项基准测试平均分从58.8（无技能）跃升至82.3（SkillOpt），实现了+23.5个百分点的绝对提升；而最佳单单元格基线平均仅76.9，使得SkillOpt比每单元格选取六种竞争方法最优结果的预言基线还高出+5.4个百分点。各基准测试相较于无技能模型的提升差值跨度从SearchQA的+9.6（无技能模型已接近上限），到SpreadsheetBench的+38.9及OfficeQA的+39.0——后两者严格的流程规范和答案格式要求暴露出零样本前沿模型的局限性。流程类基准测试提升最为显著：GPT-5.5在SpreadsheetBench上提升41.8→80.7，OfficeQA提升33.1→72.1，LiveMathematicianBench提升37.6→66.9；Qwen3.5-4B在SpreadsheetBench上提升9.3→23.9（×2.6）；GPT-5.4-nano在ALFWorld上提升34.3→69.4（×2.0）。

改进并不局限于前沿规模的目标模型。在六个基准测试中，SkillOpt平均将GPT-5.4提升了+12.7个点，GPT-5.4-mini提升+15.4个点，GPT-5.4-nano提升+26.7个点，GPT-5.2提升+16.6个点，Qwen3.5-4B提升+19.2个点，Qwen3.6-35B-A3B提升+9.1个点，平均每个模型提升约+17.6个点。小型和性能较弱的目标模型在相对意义上受益最大（例如GPT-5.4-nano在DocVQA上表现近乎翻倍，在ALFWorld上提升三倍），这与“紧凑技能组件能提供小型模型权重中尚未包含的程序性知识”的观点一致。

相同的优化接口在工具支持下执行同样有效。在Codex测试平台上，SkillOpt针对GPT-5.5的所有五项基准测试均表现最佳，相较于无技能方案平均提升+24.8个点，相比次优基线（EvoSkill）则高出+14.0点。在Claude Code测试平台中，该方案同样在GPT-5.5的五项基准上全面领先，相对无技能方案平均增益+19.1点，较EvoSkill高出+3.2点——而EvoSkill本身已使五项基准平均值从57.8提升至73.7。测试平台表格中ALFWorld对应的两栏留白，是因为ALFWorld需要持续实体环境交互，这未被标准Codex/Claude Code适配器支持；因此我们仅汇报了搜索、电子表格、文档问答、多模态问答及数学领域的测试结果。

综合来看，该表格支持了一个强有力的实证结论：无论是在直接对话还是两种工具执行框架中，跨越七个目标模型，以及在流程性和事实性基准测试上，在有限文本空间训练下优化单个紧凑技能产物，是我们所考虑的基线方法中最强的无需权重更新的适应策略。主要优势来源于反馈驱动的技能编辑，而非更优的单次提示：当先前的指令恰巧与基准匹配时，人类和大语言模型（LLM）技能可能有所帮助，但它们无法在观察到执行轨迹后修正失败；Trace2Skill无需预留验证门就能挖掘轨迹中的经验教训；TextGrad和GEPA优化的是提示而非持久的技能产物；而作为框架端最强竞争者的EvoSkill，既缺乏有界的文本学习率，也没有被拒编辑记忆功能。这些对比佐证了核心理念——保持目标模型、执行框架和评估器固定不变，仅优化可复用的技能产物。

替代解释。单细胞基线阐明了增益的驱动因素。效果并非单纯源于提示长度：人类技能已达145至516个标记长度，常常超过一次性LLM技能，但在所有直接对话模型行中均被超越，而学习到的工件仍保持紧凑（表6）。这也不仅是优化器容量的问题：即使对于GPT-5.4-nano，SkillOpt也领先于所有基线，且表5中的优化器强度分析表明，目标匹配的优化器能恢复大部分增益。最后，测试框架结果表明，该方法并非仅利用单一技能格式：EvoSkill已从Codex SpreadsheetBench单单元格中提升了

27.5 到 67.5，但 SkillOpt 额外增加了 +17.5 点 (67.5→85.0)。在程序性基准测试中收益最大，这类测试中关于工具使用和输出格式的可复用规则最为关键，但在事实性和多模态基准测试中也出现了提升。

关键数据一目了然。为方便起见，表1中的主要汇总数据如下：(i) 52/52个单元格表现最佳或并列最佳；(ii) 七个直接对话目标模型的平均每模型提升 $\approx +17.6$ 分；(iii) GPT-5.5相较于零基础的平均提升幅度：直接对话+23.5分，Codex+24.8分，Claude Code+19.1分；(iv) GPT-5.5与预言机基准的差距为+5.4分（直接对话），该数值通过SkillOpt六项基准平均分(82.3)与六种竞争方法最优值 *per cell* (76.9)之间的差值计算得出。本节余下部分将解析这些增益的成因（第4.2节）、其稳定性与可迁移性（第4.3节），以及学习产物的具体形态（第4.4节）。

4.2 消融实验

表2、图3和表3使用GPT-5.5作为目标与优化器，测试了优化器中的设计选择。总体结论是：SkillOpt受益于充足的证据、有界的文本学习率、拒绝编辑反馈以及逐轮次慢速/元更新。SearchQA由于提升空间有限，因此在多数设置下表现稳定（大部分单元格数据在 ± 1.5 分区间波动），而SpreadsheetBench和LiveMathematicianBench则揭示了学习有用程序与过度编辑技能之间的权衡关系。

证据与批次大小（图a、b、c）。图a显示程序性基准测试会奖励更多训练证据：随着优化器观察到训练分区的1→100%，SpreadsheetBench上升了47.5→78.0，LiveMathematicianBench上升了59.1→70.5，而SearchQA在仅20%时就已大致饱和在84–86左右。图b展示了相反方向上的同等稳健性：将反思小批次大小从1调整至32时，SearchQA保持在85.9–87.1范围内，SpreadsheetBench保持在75.4–77.9范围内，且默认的 $B_m=8$ 在所有三个基准测试中均处于或接近最高水平。图c在推展批次大小维度上同样表现平稳——从 $B=8$ 增至完整周期时，SearchQA维持在85.1–87.2区间，SpreadsheetBench维持在75.0–77.5区间。这些结果表明，显著的性能提升并非源于脆弱的提示搜索批次大小，而是每次更新拥有足够评分证据的真实效应。

文本学习率与调度策略（面板d、e）。面板(d)和(e)直接对比了有界文本学习与宽松设置的差异。通过扫描 $L_t \in \{1, 2, 4, 8, 16\}$ 发现，小到中等编辑预算在各场景下均具竞争力： $L_t=4$ 达到86.5/78.2/56.5，最高LiveMath分数由 $L_t=8$ 以66.9获得，而五种设置中最低分在SearchQA上仍仅为85.5。面板(e)在调度轴上验证了这一点：恒定衰减得分87.3/80.7/62.1，余弦调度87.1/77.5/61.3，线性调度87.2/72.9/62.9，说明有界更新的优势并不依赖于单一特定调度器。关键在于定性结论：任何适度的有界编辑预算均已超越无预算重写技能的基线（表3“无学习率”行，84.6/75.7/57.3）。

逐轮次的慢速/元更新（面板f，表3，图3）。慢速/元更新提供了超越当前批次的长期指导。慢速更新采样（面板f）默认设置为每轮20个示例（87.1、77.5和61.3），5、10和40个示例分别位于 ± 2.7 个点内。在匹配的默认组件行中，移除被拒编辑缓冲区使SearchQA、SpreadsheetBench和LiveMath的分数分别降低了1.6、4.6和2.4点，这支持了其作为默认循环稳定器的作用，而非额外的部署时机。慢速/元消融行的效果更为显著：同时移除元技能和慢速更新使SpreadsheetBench从77.5降至55.0（-22.5点），这是消融研究中最大的性能下降

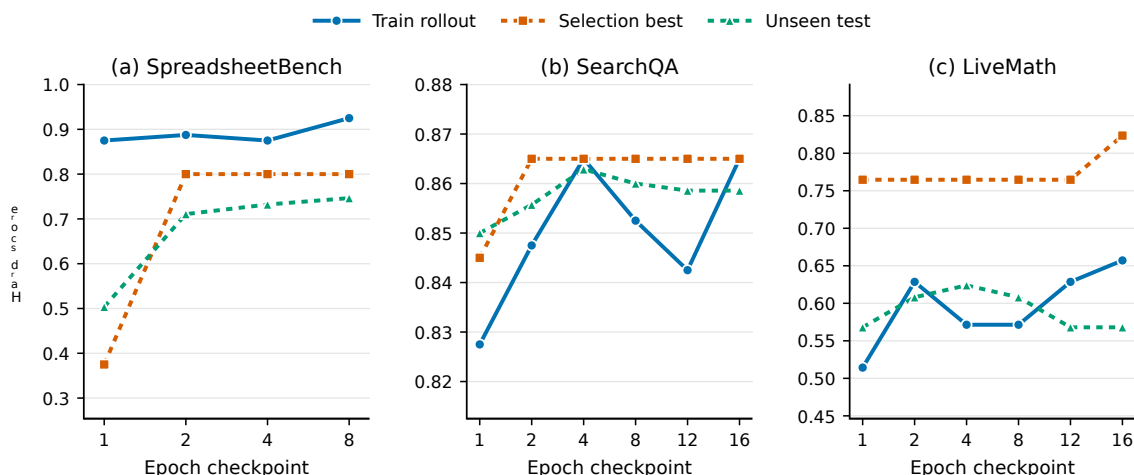


图3 三个基准测试上各epoch检查点的性能趋势：(a) SpreadsheetBench，(b) SearchQA，以及(c) LiveMath。针对每个检查点，我们报告了训练 rollout 分数、验证集上的选择最佳分数，以及在未见测试集上的最终性能。结果展示了技能质量在优化过程中的演变情况，以及验证集偏好的检查点是否与在测试集上获得最佳泛化性能的检查点相一致。

套件。图3补充了这些数值消融实验：验证检查点跟踪了跨周期的保留测试性能，证实了门控机制倾向于选择泛化能力强的技能，而非仅适应选择分割的技能。

门限严格性与编辑可观测性。验证门限被有意设置为严格标准：仅当候选技能的选择-分割分数达到当前选择分数的*strictly greater than*时才会被接受，因此同分情况将被拒绝，且已部署技能不会无声漂移。这一保守准则使得被拒编辑成为信息丰富的负面反馈，而非隐藏状态。在操作层面，每一步还会记录一个包含每次编辑接受/跳过状态的`edit_apply_report.json`，确保对`best_skill.md`的任何改动源头均可事后追溯。按周期进行的慢速/元更新会写入技能文档中受标记保护的区域——该区域不受步骤级编辑覆盖，从而将快速的周期内更新与较慢的跨周期整合分离；优化器端的元技能仅存在于教师的反思上下文中，永远不会随部署产物发布。这些实现选择解释了为何在SpreadsheetBench上同时移除元技能和慢速更新会造成特别严重的损害：此举切断了长视野证据流，也破坏了防止本地编辑覆盖持久性程序经验的保护区契约。

总体而言，消融实验表明，性能提升对具体的rollout批次、反思小批量或学习率调度相对不敏感，但对有界文本空间学习、验证门控、被拒编辑反馈以及逐轮次慢速/元更新的存在更为敏感——这些设计选择使得技能编辑行为类似于一个受控的训练循环。

4.3 分析与迁移

表4-4探讨了一个优化后的技能是否表现得像可复用的工件，而非特定任务的提示。我们测试了三种变化：将技能跨模型规模部署（表4）、在不同执行框架间迁移（表4），以及将其应用于相近的数学基准测试，包括OlympiadBench [41]和Omni-MATH [14]（表4）。随后，表5通过用与部署模型规模匹配的目标优化器替代前沿优化器，检验了增益中有多少依赖于优化器容量。

跨模型迁移。表4(a)报告了四项*genuine*跨模型迁移实验（GPT-5.4→GPT-5.4条目仅作为域内参考保留，并在迁移列标记为-，因为源与目标模型相同）：基于GPT-5.4训练的SpreadsheetBench技能可迁移至GPT-5.4-mini（+9.4）和GPT-5.4-nano（+3.0），LiveMath技能可迁移至GPT-5.4-mini（+4.5）和GPT-5.4-nano（+5.6）。所有四项跨模型迁移结果均为正向，其中一项的迁移技能*surpasses*甚至超过域内SkillOpt基准（LiveMath GPT-5.4-nano：28.8迁移成绩 vs. 27.2域内成绩），表明部分学习流程具有目标模型无关性。其余跨模型迁移仍保留了域内增益的有效部分——例如SpreadsheetBench GPT-5.4-mini保留了82%域内增益（+9.4 / +11.4）——且所有结果均高于目标模型的零技能基线。

跨工具迁移。表4(b)中的工具切换行是最明显的部署信号。在Codex循环内训练的SpreadsheetBench技能迁移至Claude Code时，相比Claude Code无技能基线（22.1→81.8）实现了+59.7的绝对增益，略超Claude Code领域内SkillOpt参考值80.4；而对称的Claude-Code→Codex迁移则在Codex基线（27.5→71.1）基础上提升+43.6。在LiveMath上，Codex→Claude Code迁移幅度较小（40.8基线上提升+1.6）但仍为正值，而Claude-Code→Codex迁移则增加+12.8（35.2→48.0）。由于两套工具暴露不同的工具/文件API和命令接口，这些正向迁移表明所学规则不仅限于工具特定命令配方。尤其在SpreadsheetBench中，迁移技能似乎编码了工作簿级操作流程，如结构优先检查、公式感知验证和静态值物化，因此在一个执行环境中优化技能的成本可分摊至相关部署环境。

跨基准迁移。跨基准迁移是三种转移中最严格的一种：源基准和目标基准仅共享广泛的类别（数学）。在表4(c)报告的OlympiadBench→Omni-MATH方向上，所迁移的技能在我们评估的所有三种模型规模上均呈现正向效果，GPT-5.4提升了+3.7，GPT-5.4-mini提升了+1.8，GPT-5.4-nano提升了+1.3。这些数值虽小于领域内和跨工具链迁移的结果——这并不意外，因为它们要求优化后的技能在测试实例和答案格式规范均发生变化后，仍能保留有用的程序性知识——但结果始终保持正向，支持了预期解释：优化后的技能编码的是可复用的数学程序，而非记忆的基准特定格式。

优化器强度的影响。由于SkillOpt中的优化器仅在离线训练循环期间运行，在部署时从不调用，因此优化器的选择是一个训练时的杠杆：更强的优化器可以提升部署技能的效能，而不会增加使用该技能的推理成本。部署产物仍是一个静态的best_skill.md，仅调用目标模型。表5通过运行相同的循环，采用两种优化器方案——一个强大的前沿优化器（GPT-5.5）和一个与目标模型共享的匹配优化器——量化了这一杠杆效应，同时保持滚动批次、验证门限、有限编辑预算、被拒编辑缓冲及慢速/元更新不变。

从中可以得出两点观察。*First*，更强的优化器在我们测试的每一个（基准，目标）单元上都产生了更大的绝对增益：GPT-5.4-nano在SpreadsheetBench上提升了+19.0对比+11.9，在SearchQA上提升了+19.0对比+14.1；而GPT-5.4-mini也遵循相同的顺序（在SpreadsheetBench上+11.4对比+7.1，在SearchQA上+4.3对比+2.4）。有界编辑、验证门控的循环正是这种单调性的保证：若没有这层门控，更强的优化器同样可能推动更大但有害的重写。*Second*，目标匹配的优化器远未失效——它在四个单元中恢复了强优化器增益的56%至74%，这证实了SkillOpt并非简单地将强教师模型的知识蒸馏到弱学生模型中：优化循环本身贡献显著

Benchmark	Target	Baseline	Strong optimizer (GPT-5.5)	Target-matched optimizer
SpreadsheetBench	GPT-5.4-mini	36.1	47.5 _{+11.4}	43.2 _{+7.1}
	GPT-5.4-nano	23.5	42.5 _{+19.0}	35.4 _{+11.9}
SearchQA	GPT-5.4-mini	75.9	80.2 _{+4.3}	78.3 _{+2.4}
	GPT-5.4-nano	55.8	74.8 _{+19.0}	69.9 _{+14.1}

表5 优化器强度的影响。每个（基准，目标）组合由强前沿优化器（GPT-5.5，加粗）或与目标模型相匹配的优化器进行优化；SkillOpt循环中的其他所有内容均保持不变。相较于目标无技能基线的增益以绿色小下标显示；同一行中两种优化器设置使用相同的基线。优化器仅在离线训练期间运行，因此强优化器列在部署时不增加任何成本。

Benchmark	Initial (tok)	Final (tok)	Edits	Train tokens	Cost / pt
SearchQA	16	857	4	213.8M	37.9M
SpreadsheetBench	224	1,995	4	21.4M	0.6M
OfficeQA	145	883	1	20.8M	1.1M
DocVQA	81	959	3	188.2M	46.4M
LiveMath	154	379	1	23.2M	3.6M
ALFWorld	516	1,321	2	59.3M	15.9M

表6 GPT-5.5 / GPT-5.5（学生/教师）技能运行的成本与编辑经济性。初始与最终best_skill.md长度以标记为单位；编辑次数为接受的边界更新数量；成本/点为每绝对测试点增益所需的训练标记数。每个基准测试的一个代表性学习规则如图4所示。

在优化器已有能力的基础上带来显著价值。实际应用中，只要条件允许，高容量的前沿优化器就是理想的默认选择——它仅需训练时的API调用，不会增加部署成本；而如果预算有限，转而使用与目标匹配的优化器，同样的流程依然有效。

4.4 习得技能：紧凑性、成本及示例

SkillOpt的一个核心前提是，可训练对象应保持为一个小型、可检查的文本文档。表1-5展示了该优化器的有效性；本小节将探讨其输出的实际形态及其成本。我们从三个维度——紧凑性、编辑经济性和单位成本效益——对学习产物进行特征描述，随后为每个基准展示一条具有代表性的学习规则，以此说明在有限更新循环中存活下来的程序性知识类型。

紧凑性。最终技能的规模均较小。在表6的六个基准测试中，最终的best_skill.md范围从379个令牌（LiveMathematicianBench）到1,995个令牌（SpreadsheetBench），中位数约为920个令牌。即使最长的习得技能也远低于现代前沿模型的典型系统提示预算，最短的甚至能在一个屏幕内完整显示。从初始技能到最终技能的增长幅度适中（ $\times 2.5$ 至 $\times 53$ ，取决于初始技能是单行还是段落形式），但最终令牌数的绝对规模足够小，领域从业者能在几分钟内阅读、审核并修改部署的成果物。

编辑经济。第二个显著模式是，收益来自于极少被采纳的编辑。在六个基准测试中，实际优化过程中提交至best_skill.md的编辑数量介于1到4次之间（中位数2.5）。LiveMathematicianBench相比无技能状态取得的+29.3分提升源自single次被采纳的编辑，而OfficeQA的+39.0分提升同样仅需一次采纳的编辑。这直接验证了验证关卡的实际作用：优化模型每轮会提出更多编辑，但仅有极少数能通过留出检验，最终融入部署技能。因此，优化器在文本空间中的大部分搜索尝试均被拒绝

SearchQA。“从线索措辞推断预期答案类型，然后选择由共现的独特证据支持的最短规范实体。” SpreadsheetBench。“检查工作簿结构和公式，然后在完整请求的目标范围内写入评估后的静态值，而非依赖Excel重新计算。” OfficeQA。“将解析后的页面视为主要证据，锁定表格/日期/单位上下文，并精确输出请求的舍入值，不含额外标签。” DocVQA。“对于表格、表单、图表和图例，首先将问题绑定到确切的视觉行/标题/字段，然后仅复制对齐的答案片段。” LiveMathematicianBench。“在最强陈述的多选题中，根据定理强度对选项排序，优先选择有依据的更强结果选项而非正确但较弱的推论。” ALFWorld。“保持一个视野感知的已访问/前沿记录，在同类重复失败后多样化搜索，并在持有目标前避免重返目的地。”

图4展示了从表6中GPT-5.5/GPT-5.5运行的最终`best_skill.md`提取的代表性学习规则，每个基准各一条。每条规则均直接引自实际部署的技能。值得注意的是，所有规则均为流程性而非针对特定实例，且其中几条编码了前沿模型零样本情况下未应用的纪律形式（答案格式化、证据绑定、搜索前沿管理）。

被拒绝编辑缓冲区（第3.5节）保存以供将来使用，并且永远不会到达目标模型。部署的技能因此是紧凑的，而非所有反思的集合。

测试集每点增益的成本。训练令牌列量化了运行循环的成本。可见两种模式：程序性基准测试——如SpreadsheetBench、OfficeQA、LiveMathematicianBench——其推演过程短且成本低，达到每绝对测试集点数消耗0.6–3.6百万训练令牌，尽管这些基准上的绝对增益最大（例如OfficeQA上每点1.1百万令牌，总计20.8百万令牌，增益达+39.0点）。而轨迹较长或具有更丰富多模态上下文的基准测试——SearchQA（37.9百万/点）和DocVQA（46.4百万/点）——每点成本高出一个数量级。重要的部署区别在于，这一成本仅在技能训练期间支付一次；导出后，优化后的`best_skill.md`不会增加优化器调用或权重更新，仅向目标代理添加一个紧凑的文本工件。

学到的技能究竟表达了什么？图4重现了每个基准测试中一条具有代表性的学习规则，这些规则直接摘自表6中各案例研究的最终`best_skill.md`版本。有三点观察尤为突出。首先，这些规则呈现为*procedural*而非*instance-specific*：无一涉及具体问题、文件或实体名称。其次，它们始终贯彻着前沿模型在零样本情况下所缺乏的规范约束：答案格式要求（OfficeQA、LiveMathematicianBench）、证据与特定视觉区域的绑定（DocVQA）、工作簿结构优先推理（SpreadsheetBench）、搜索边界纪律（ALFWorld），以及规范实体选择（SearchQA）。最后，这些规则读起来像是深思熟虑的人类从业者接触基准测试一天后所写的——但它们实际上是由优化器自动生成，并通过逐条编辑在保留数据上验证的。

启示。上述四点观察共同支持了中心主张的一个更强版本。紧凑性（< 2,000个标记）与编辑经济性（1-4次采纳的编辑）意味着部署的产物是可解释的。每点成本（0.6M–46.4M标记/点）表明训练成本是可量化的，并在部署前已支付。学习到的规则形态——程序化、可泛化且与深思熟虑的人类实践者所编写的规则一致——证明了有限更新与验证门控的文本空间优化能够发现可迁移的程序性知识，而非仅仅对训练集过拟合。这补充了4.3节中跨模型、跨工具链与跨基准测试的迁移证据：该产物之所以能迁移，是因为其编码的许多规则本身具有可迁移性。

4.5 定性技能演进

我们通过考察两个代表性运行案例，来理解优化后的技能实际学到了什么。在ALFWorld案例中，使用GPT-5.4-nano作为学生模型，GPT-5.5作为教师模型；而SpreadsheetBench案例则采用GPT-5.5同时作为冻结学生模型和优化器模型。在这两种情况下，SkillOpt都未用无关提示替换初始技能，而是通过采纳的编辑，围绕在轨迹推演中观察到的重复失败模式，添加了紧凑的流程约束。

ALFWorld. 初始的ALFWorld技能提供了一个通用的家务执行方案：搜索目标物品，拾取它，必要时进行转换，然后放置到目的地。经过认可的编辑使该方案更具状态感知能力，减少了循环倾向。优化后的技能学会了精确的对象名称匹配，因此诸如马克杯、杯子、平底锅和壶等相关物品不会相互替代。它增加了已访问位置记忆功能，使得未检查过的容器和台面会被优先考虑，而非反复查看可能但已穷尽的位置。此外，它还引入了目的地记忆、双步进度锁定以及直接完成规则：一旦代理能够清洁、加热、冷却、放置或以其他方式完成下一个子目标，就应采取该可行动作，而非再次检查、关闭或验证。从定性角度看，该技能从通用的搜索-转换-放置策略，进化为一个具备对象身份识别、搜索记忆、进度锁定和循环中断机制的有限状态执行策略。在此代表性测试中，所选技能将ALFWorld的保留测试性能从49.3提升至74.6。

SpreadsheetBench. 初始的SpreadsheetBench技能已指导代理使用Python电子表格库并保留无关的工作簿内容。采纳的编辑将这一通用自动化流程转变为工作簿取证策略。优化后的技能学会检查实际工作簿而非依赖预览，跨多张工作表定位表头和目标范围，在执行查找或聚合前规范化键和单元格类型，并在结构编辑时保留格式。它还新增了一条针对公式提示的关键规则：当评分器读取单元格值时，代理应计算并写入评估后的静态值，即使提示中提及如INDEX/MATCH或XLOOKUP的公式。后续编辑进一步要求填充完整的目标范围（包括当前为空的结果单元格），将辅助计算保留在Python中而非添加工作簿工件，以及重新打开已保存的工作簿以检查边界行和剩余空白项。在此代表性测试中，所选技能将SpreadsheetBench的保留测试性能从40.4提升至78.9。

5 结论

我们提出了SkillOpt，这是一种文本空间优化器，将外部技能文档视为冻结大型语言模型（LLM）代理的可训练状态。通过将执行任务的目标模型与编辑技能的优化器分离，并采用有限编辑预算、小批量反思、保留验证门、拒绝编辑缓冲区和逐轮慢速/元更新机制，SkillOpt将技能提升转化为受控学习过程，而非临时性提示修改。在六个基准测试、七个目标模型和三种执行模式下，SkillOpt在全部52个评估单元中表现最佳或并列最佳，使GPT-5.5在直接对话中较无技能状态平均提升+23.5分，在Codex和Claude Code测试框架下分别提升+24.8/ +19.1分，并以平均+5.4分的优势超越人工、LLM、Trace2Skill、TextGrad、GEPA和EvoSkill等各单元最强基线。分基准案例分析表明，这些增益源自仅需1-4次采纳编辑即可构建的紧凑型（<2,000词元）可解释技能构件，且部署后的技能可跨模型规模、测试框架及相邻基准迁移。这些结果表明，紧凑的自然语言技能可作为前沿代理的实用领域适配层，在不修改模型权重的情况下实现可复用的性能提升。

展望。SkillOpt针对单一目标领域优化单一技能构件；自然的扩展方向包括跨领域共享基础设施的技能库、在基准测试间复用优化器端的元技能、为开放式任务设计无奖励或偏好驱动验证关卡，以及将优化后的技能自蒸馏回目标模型作为权重级适应的基石。我们期望将技能本身视为可训练对象——而非提示的附属产物——能让未来研究将完整的优化工具包（学习率、调度、正则化、课程学习、验证）应用于迄今仍需人工设计的智能体组件层面。

参考文献

- [1] 姚顺宇, Jeffrey Zhao, 余典, 杜楠, Izhak Shafran, Karthik Narasimhan, 和曹原。React: 语言模型中推理与行动的协同。 *arXiv preprint arXiv:2210.03629*, 2022。
- [2] 蒂莫·希克, 简·德维迪-尤, 罗伯托·德西, 罗伯塔·雷勒阿努, 玛丽亚·洛梅利, 埃里克·汉布罗, 卢克·泽特勒迈尔, 尼古拉·坎塞达, 与托马斯·夏洛姆。Toolformer: 语言模型可自学使用工具。 *Advances in neural information processing systems*, 36卷: 68539–68551页, 2023年。
- [3] 王冠之、谢雨琪、姜云帆、Ajay Mandlekar、肖超威、朱玉可、范林溪与Anima Anandkumar。Voyager: 基于大型语言模型的开放式具身智能体。 *arXiv preprint arXiv:2305.16291*, 2023年。
- [4] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, 和 Ofir Press。Swe-agent: 智能体-计算机接口实现自动化软件工程。 *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024。
- [5] 陈润阳, 王学志, 卢逸风, 刘瀚霄, Quoc V Le, 周登尼, 陈新云。大语言模型作为优化器。发表于 *The Twelfth International Conference on Learning Representations*, 2023年。
- [6] Omar Khattab、Arnav Singhvi、Paridhi Maheshwari、张致远、Keshav Santhanam、Sri Vardhamanan、Saiful Haq、Ashutosh Sharma、Thomas T Joshi、Hanna Moazam等。Dspy: 将声明式语言模型调用编译为自我改进的流水线。 *arXiv preprint arXiv:2310.03714*, 2023年。
- [7] 李湘怡, 陈文博, 刘益民, 郑深翰, 陈晓坤, 何一峰, 李雨波, 游炳然, 沈浩天, 孙建凯等。Skillsbench: 评估智能体技能在多样化任务中的表现基准。 *arXiv preprint arXiv:2602.12670*, 2026。
- [8] 姜雅娜, 李德龙, 邓海宇, 马柏鹤, 王旭, 王琴, 余广晟。Sok: 智能体技能——超越大语言模型代理的工具使用。 *arXiv preprint arXiv:2602.20867*, 2026。
- [9] 倪经纬、刘一豪、刘鑫鹏、孙宇涛、周梦雨、程鹏宇、王德新、姜小溪、姜冠军。Trace2skill: 将轨迹局部经验提炼为可迁移的智能体技能。 *arXiv preprint arXiv:2603.25158*, 2026年。
- [10] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen 和 Tu Vu。Evoskill: 面向多智能体系统的自动化技能发现。 *arXiv preprint arXiv:2603.02766*, 2026。
- [11] 刘星岩, 罗西月, 李林瑜, 黄港宏, 刘剑锋, 乔鸿林。Skillforge: 在云技术支持中锻造领域专用、自我进化的智能体技能。 *arXiv preprint arXiv:2604.08618*, 2026年。

[12] 沈帅科、程文铎、马明谦、Alistair Turcan、张金烨与马剑。Skillfoundry: 从异构科学资源构建自我进化的智能体技能库。 *arXiv preprint arXiv:2604.03964*, 2026年。

[13] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang等。Gepa: 反思式提示进化可超越强化学习。 *arXiv preprint arXiv:2507.19457*, 2025。

[14] 高博飞, 宋非凡, 杨哲, 蔡泽帆, 苗一博, 董清修, 李磊, 马成浩, 陈亮, 许润新, 唐正阳, 王本有, 赞道广, 全上豪然, 葛征, 沙磊, 张一畅, 任宣成, 刘天宇, 常宝宝。Omni-math: 面向大语言模型的通用奥林匹克数学基准, 2024。URL {v*}。

[15] 宋伟佳、岳佳树、庞哲。Abstral: 通过迭代优化与拓扑设计自动构建多智能体系统。 *arXiv preprint arXiv:2603.22791*, 2026。

[16] 何宇飞、刘俊成、刘越、李一博、曹拓、胡致远、徐新星与Bryan Hooi。Evotest: 进化式测试时学习赋能自进化智能体系统。 *arXiv preprint arXiv:2510.13220*, 2025年。

[17] 杨雨涛, 李俊松, 潘千钧, 詹碧浩, 蔡宇轩, 杜琳, 周杰, 陈凯, 陈琴, 李鑫等。Autoskill: 通过技能自我进化的经验驱动终身学习。 *arXiv preprint arXiv:2603.01145*, 2026。

[18] 陈曦、王卓云、谢欣、姚武冠男、方润楠、乔硕飞、曹可欣、郑国洲、齐翔、张鹏等。Skillx: 自动化构建智能体技能知识库。 *arXiv preprint arXiv:2604.04804*, 2026。

[19] 方润南, 梁远, 王孝彬, 吴佳隆, 乔硕斐, 谢鹏军, 黄斐, 陈华钧, 张宁豫。Memp: 探索智能体程序性记忆。 *arXiv preprint arXiv:2508.06433*, 2025。

[20] 张翰荣, 范世成, 邹亨利, 陈彦凯, 王振廷, 周佳玉, 李承泽, 黄伟杰, 姚一飞, 郑克宁等。Evoskills: 通过协同进化验证实现自我进化的智能体技能。 *arXiv preprint arXiv:2604.01687*, 2026。

[21] 马子玉, 杨士东, 纪宇翔, 王旭聪, 王勇, 胡一鸣, 黄同文, 褚祥祥。SkillClaw: 让技能与代理进化器共同进化。 *arXiv preprint arXiv:2604.08377*, 2026年。

[22] 彭夏, 陈建文, 王汉阳, 刘嘉琪, 曾凯德, 王宇, 韩思伟, 周毅阳, 赵旭江, 陈海峰等。Skillrl: 通过递归技能增强的强化学习进化智能体。 *arXiv preprint arXiv:2602.08234*, 2026。

[23] 王炯晓、严乔静、王亚伟、田一君、Soumya Smruti Mishra、徐志超、Megha Gandhi、徐盼盼、张琳丽。基于技能库自我提升的强化学习智能体。 *arXiv preprint arXiv:2512.17102*, 2025年。

[24] 邱立斌, 高志荣, 陈俊福, 叶宇航, 黄炜智, 薛晓波, 邱文凯, 唐硕。Autorefine: 从轨迹到可复用专长——持续LLM智能体精进之道。 *arXiv preprint arXiv:2601.22758*, 2026。

- [25] 米启瑞, 马志坚, 杨梦月, 李浩轩, 王一森, 张海峰, 王俊。Procmem: 通过非参数化PPO从经验中学习可重用程序记忆的LLM智能体方法。 *arXiv preprint arXiv:2602.01869*, 2026。
- [26] 吴荣, 王晓曼, 梅建标, 蔡品龙, 付道成, 杨程, 温立成, 杨雪萌, 沈宇帆, 王雨欣等。Evolver: 通过经验驱动生命周期实现自我进化的LLM智能体。 *arXiv preprint arXiv:2510.16079*, 2025。
- [27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, 和 Shunyu Yao。反思: 基于言语强化学习的语言智能体。 *Advances in neural information processing systems*, 36:8634–8652, 2023年。
- [28] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang等。自我优化: 基于自我反馈的迭代优化。 {v*}, 36:46534–46594, 2023年。
- [29] 马修·邓恩、莱文特·萨贡、迈克·希金斯、V·乌古尔·居内伊、沃尔坎·奇里克和Kyunghyun Cho。SearchQA: 一个通过搜索引擎上下文增强的新型问答数据集。 *arXiv preprint arXiv:1704.05179*, 2017年。
- [30] 马泽尧, 张博涵, 张静, 余继繁, 张晓康, 张晓晗, 罗思佳, 王曦, 唐杰。Spreadsheetbench: 面向真实世界具有挑战性的电子表格操作。 *Advances in Neural Information Processing Systems*, 37: 94871–94908, 2024。
- [31] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen等。Officeqa pro: 面向端到端基础推理的企业级基准测试。 *arXiv preprint arXiv:2603.08655*, 2026。
- [32] Minesh Mathew, Dimosthenis Karatzas, 和 CV Jawahar。DocVQA: 一种面向文档图像的视觉问答数据集。收录于 *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, 第2200–2209页, 2021年。
- [33] 林阳赫、齐耀宇、韩泽东、廖宝浩、徐新星、Micah Goldblum、边江与Nima Mesgarani。《LiveMathematicianBench: 基于证明草稿的数学家级推理实时基准》, 2026年。URL <https://arxiv.org/abs/2604.01754>。
- [34] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, 和 Matthew Hausknecht。{ALFW}orld: 对齐文本与具身环境以进行交互式学习。载于 *International Conference on Learning Representations*, 2021年。网址<https://openreview.net/forum?id=0IOX0YcCdTn>。
- [35] OpenAI。GPT-5.4发布公告, 2026年3月。URL <https://openai.com/index/introducing-gpt-5-4/>。
- [36] Qwen团队。Qwen3.5: 迈向原生多模态智能体, 2026年2月。URL <https://qwen.ai/blog?id=qwen3.5>。
- [37] Qwen团队。Qwen3.6-35B-A3B: 面向所有人开放的智能编码能力, 2026年4月。URL <https://qwen.ai/blog?id=qwen3.6-35b-a3b>。
- [38] OpenAI。Codex: 一款基于云的软件工程代理, 2025年。网址 <https://openai.com/index/introducing-codex/>。访问日期: 2026-05-06。

[39] Anthropic. Claude代码：一种AI编码代理系统，2025年。网址<https://www.anthropic.com/claude-code>。访问日期：2026-05-06。

[40] Mert Yuksekgonul、Federico Bianchi、Joseph Boen、Sheng Liu、Zhi Huang、Carlos Guestrin 和 James Zou。Textgrad：通过文本实现自动“微分”。*arXiv preprint arXiv:2406.07496*, 2024年。
[41] 何超群、罗人杰、白雨卓、胡圣鼎、泰振冷、沈俊浩、胡金一、韩旭、黄宇杰、张宇翔等。Olympiadbench：一个具有挑战性的基准，旨在通过奥林匹克级别的双语多模态科学问题促进AGI发展。*arXiv preprint arXiv:2402.14008*, 2024年。

A 额外方法详情与优化器提示

本附录详细阐述了SkillOpt的可执行细节。优化循环保持任务执行模型固定不变，仅针对文本技能文档进行训练。一个独立的优化器模型会读取推演证据，提出补丁式编辑建议，合并并排序这些编辑，随后将每个候选技能提交至预留的选择门控环节。任务执行模型仅接收当前技能与基准任务；它不会看到下方优化器的提示信息。

B 局限性

SkillOpt研究技能优化作为模型权重调整的轻量级替代方案，但仍存在若干实际限制。首先，优化循环依赖于评分轨迹和预留选择分割，因此最直接适用于目标任务具备自动验证器、精确匹配指标、可执行检查或其他可靠反馈信号的场景。对于成功标准主观、多维或评估成本高昂的开放式领域，验证门限可能需要更强的人工或基于模型的评估。其次，尽管部署产物仅是紧凑的 $\{v^*\}$ ，但训练技能需要额外的展开计算和优化器模型调用；当同一技能被复用该成本可分摊，但对一次性任务可能吸引力不足。第三，SkillOpt有意优化单一可移植技能而非扩展大型技能库或更改模型权重。该设计提升了部署简便性，但单一技能可能难以满足需要大量不相关程序的高度异质领域。最后，优化后的技能编码来自训练分布的领域特定启发式，因此在将其迁移至显著不同的模型、框架或任务设置前，严格的预留评估仍属必要。

C 实验协议详情

基准测试与评估指标。我们采用各基准自带的评估器，针对预留测试样本报告硬性成功率或精确匹配准确度。SearchQA衡量抽取式问答能力；SpreadsheetBench评估面向电子表格的代码与工具使用；OfficeQA和DocVQA测试本地文档及多模态文档推理；SealQA侧重噪声检索场景；LiveMathematicianBench评估数学选择题推理；ALFWorld则检验序列决策能力。基于数据集的基准测试采用确定性的训练/筛选/测试划分比例，若基准未指定划分方式则默认按2:1:7分配。筛选集仅用于候选技能间的模型选择；所有核心指标均基于预留测试数据计算得出。

基线方法。无技能基线评估冻结学生模型时未使用优化技能文档。人类技能和LLM技能基线则分别采用人工编写和单次生成的技能，在

相同的评估协议。Trace2Skill从训练轨迹中挖掘技能构件，并在没有SkillOpt迭代验证门的情况下评估冻结的学生模型。TextGrad和GEPA是针对直接对话设置的反思式提示优化基线。EvoSkill被纳入基于测试框架的比较中，前提是有匹配的完整运行记录可用。未按照最终对齐协议测量的条目标记为-而非与不兼容的运行结果混合。

优化协议。除非另有说明，SkillOpt默认运行四个周期，其参数配置如下：rollout批次大小为40，反射小批次大小为8，文本学习率为4，采用最小速率为2的余弦学习率衰减策略，启用保留验证门控机制，慢速更新模式下采样20个示例，并启用优化器端元技能功能。优化器会分别分析成功与失败案例，提出补丁式技能修改建议，合并重复或矛盾的提案，在当前学习率上限下对修改建议进行排序，并应用选定修改以形成候选技能。该候选技能需在选拔集上评估，仅当提升当前选拔分数时才会被采纳；最终导出的最优技能标记为{v*}。优化过程中，学生模型、后端框架、测试工具及基准评估器均保持固定不变。

消融实验方案。单因素消融实验通过改变单个标量或组件，同时保持优化器配置的其余部分不变。训练规模消融实验将训练/选择/测试的分割比例固定为2:1:7，并调整优化器暴露于训练分区的比例。100%行表示在相同分割比例下使用完整的训练分区，因此可直接与较小子集行进行比较。组件消融实验逐一移除或修改某一机制，包括编辑预算、被拒编辑缓冲区以及逐轮次的慢速/元更新。

C.1 优化过程

算法1扩展了实验中使用的流程。核心状态变量包括当前技能 s_{cur} 、最佳验证门控技能 s_{best} 、选择分数缓存 C 、包含被拒绝编辑和观察到的失败模式的步骤缓冲 B ，以及仅用于指导未来编辑生成的优化器侧元技能 m_{metao}

C.2 优化器提示合约

以下模块重现了优化器模型所使用的操作提示契约，术语已统一为论文中优化器/训练模型的框架。这些提示要求输出JSON格式，以便编辑能够无需人工干预即可被解析、筛选、应用及验证。

C.2.1 故障分析：analyst_error.md

You are an expert failure-analysis agent for AI agent tasks.

You will be given MULTIPLE failed agent trajectories from a single minibatch and the current skill document.

Your job is to identify the most important COMMON failure patterns across the batch and propose a concise set of skill edits.

Analysis Process

1. Read ALL trajectories in the minibatch.
2. Identify the most prevalent, systematic failure patterns across them.
3. For each pattern, classify its failure type.
4. Propose skill edits that address the COMMON patterns, not individual edge cases.
5. Edits must be generalizable; do not hardcode task-specific values.
6. Only patch gaps in the skill; do not duplicate existing content.

You will be told the maximum number of edits (the budget L). Produce AT MOST L edits,

Algorithm 1 SKILLOPT skill optimization

要求: 冻结训练模型 M , 优化器模型 O , 训练框架 h , 数据分割 $D_{\text{train}}, D_{\text{sel}}, D_{\text{test}}$, 初始技能 s_0 , 训练轮次 E , 编辑预算调度 L_t , 推展批次大小 B , 累积因子 A , 反思小批次大小 B_m 确保: 最佳验证门控技能 s_{best} 和保留测试分数

- 1: $s_{\text{cur}} \leftarrow s_0, s_{\text{best}} \leftarrow s_0, \mathcal{C} \leftarrow \emptyset, \mathcal{B} \leftarrow []$, $m_{\text{meta}} \leftarrow \emptyset$
- 2: 分数_{cur} $\leftarrow$ 评估($M, h, s_0, D_{\text{sel}}$); 分数_{best} $\leftarrow$ 分数_{cur}
- 3: $\mathcal{C}[\text{哈希}(s_0)] \leftarrow$ 分数_{cur}
- 4: 对于 $e =$ 从1到 E 执行
- 5: 将 D_{train} 随机分配到推展批次; 重置 $\mathcal{B} \leftarrow []$
- 6: 对于训练轮次 e 中的每个优化步骤执行
- 7: 通过为采样任务 x 执行 $h(M, x, s_{\text{cur}})$ 收集 A 个推展批次
- 8: 将推展证据分为失败和成功, 然后分割为大小为 B_m 的小批次
- 9: 要求 O 分析失败小批次并生成失败补丁提案
- 10: 要求 O 分析成功小批次并生成成功补丁提案
- 11: 要求 O 合并失败提案、合并成功提案, 并执行最终的失败优先合并
- 12: 要求 O 对合并后的编辑进行排序并保留最多 L_t 个编辑
- 13: 应用选定的编辑以获得候选技能 $\tilde{s}$
- 14: 如果 哈希($\tilde{s}$) $\in \mathcal{C}$ 则
- 15: 分数_{cand} $\leftarrow \mathcal{C}[\text{哈希}(\tilde{s})]$
- 16: 否则
- 17: 分数_{cand} $\leftarrow$ 评估($M, h, \tilde{s}, D_{\text{sel}}$)
- 18: $\mathcal{C}[\text{哈希}(\tilde{s})] \leftarrow$ 分数_{cand}
- 19: 结束如果
- 20: 如果 分数_{cand} $>$ 分数_{cur} 则
- 21: $s_{\text{cur}} \leftarrow \tilde{s}$; 分数_{cur} $\leftarrow$ 分数_{cand}
- 22: 如果 分数_{cand} $>$ 分数_{best} 则
- 23: $s_{\text{best}} \leftarrow \tilde{s}$; 分数_{best} $\leftarrow$ 分数_{cand}
- 24: 结束如果
- 25: 否则
- 26: 将拒绝的编辑和观察到的失败模式添加到 $\mathcal{B}$
- 27: 结束如果
- 28: 结束对于
- 29: 如果 $e \geq 2$ 且慢速更新启用 则
- 30: 比较前一和当前轮次结束技能下的相同采样任务
- 31: 要求 O 提供受保护的纵向指导; 通过 D_{sel} 验证注入的指导
- 32: 结束如果
- 33: 如果 $e \geq 2$ 且优化器内存启用 则
- 34: 要求 O 更新 m_{meta} 用于未来的编辑生成和选择
- 35: 结束如果
- 36: 结束对于
- 37: 分数_{test} $\leftarrow$ 评估($M, h, s_{\text{best}}, D_{\text{test}}$)
- 38: 返回 s_{best} , 分数_{test}

focusing on the highest-impact patterns. You may produce fewer if warranted.

Respond ONLY with a valid JSON object (no markdown fences, no extra text):

```
{
  "batch_size": <number of trajectories analysed>,
  "failure_summary": [
    {"failure_type": "<type>", "count": <int>, "description": "<one-line>"}
  ],
}
```

```

"patch": {
  "reasoning": "<why these edits address the batch's common failures>",
  "edits": [
    {"op": "append",          "content": "<markdown to add at end of skill>"},
    {"op": "insert_after", "target": "<exact heading/text to insert after>",
     "content": "<markdown>"},
    {"op": "replace",        "target": "<exact text to replace>",
     "content": "<replacement>"},
    {"op": "delete",         "target": "<exact text to remove>"}
  ]
}

```

Only include edits that are needed. "edits" can be an empty list if no patch is warranted.

IMPORTANT: The skill document may contain a section between
 <!-- SLOW_UPDATE_START --> and <!-- SLOW_UPDATE_END --> markers.
 This is a PROTECTED section managed by a separate slow-update process.
 Do NOT propose any edits that target, modify, or delete content within these markers.

C.2.2 成功分析: `analyst_success.md`

You are an expert success-pattern analyst for AI agents.

You will be given MULTIPLE successful agent trajectories from a single minibatch and the current skill document. Your job is to identify generalizable behavior patterns that are COMMON across the batch and worth encoding in the skill.

Rules

- Only propose patches for patterns NOT already covered in the skill.
- Focus on patterns that appear across MULTIPLE trajectories in the batch.
- Be concise. Patterns must generalize beyond specific tasks.
- Prefer reinforcing existing sections over adding new top-level sections.

You will be told the maximum number of edits (the budget L). Produce AT MOST L edits, focusing on the most broadly applicable patterns. You may produce fewer if warranted.

Respond ONLY with a valid JSON object:

```

{
  "batch_size": <number of trajectories analysed>,
  "success_patterns": ["<pattern 1>", "<pattern 2>"],
  "patch": {
    "reasoning": "<why these patterns are worth encoding>",
    "edits": [
      {"op": "append",          "content": "<markdown>"},
      {"op": "insert_after", "target": "<heading/text>", "content": "<markdown>"},
      {"op": "replace",        "target": "<old text>",      "content": "<new text>"},
      {"op": "delete",         "target": "<exact text to remove>"}
    ]
  }
}

```

"edits" may be empty if the skill already covers all observed patterns.

IMPORTANT: The skill document may contain a section between
 <!-- SLOW_UPDATE_START --> and <!-- SLOW_UPDATE_END --> markers.
 This is a PROTECTED section managed by a separate slow-update process.
 Do NOT propose any edits that target, modify, or delete content within these markers.

C.2.3 故障合并: `merge_failure.md`

You are a skill-edit coordinator. You receive multiple independently-proposed patches from FAILURE analysis of agent trajectories. Merge them into ONE coherent,

non-redundant patch.

Merge guidelines:

1. Deduplicate: keep the best-worded version of similar edits.
2. Resolve conflicts: if patches contradict on the same point, choose the one with stronger justification or synthesize both.
3. Preserve unique insights: include all non-redundant corrective edits.
4. Prevalent-pattern bias: edits appearing consistently across multiple patches address systematic failures; preserve them with HIGH priority. Edits from only one patch may be discarded if task-specific.
5. Independence: no two edits in the merged patch may target the same text region.
6. Support count: for each merged edit, estimate how many source patches support it.
7. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary of key consolidation decisions>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if insert_after or replace or delete>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "failure"
    }
  ]
}
```

C.2.4 成功合并: `merge_success.md`

You are a skill-edit coordinator. You receive multiple independently-proposed patches from SUCCESS analysis of agent trajectories. Merge them into ONE coherent patch that reinforces effective patterns.

Merge guidelines:

1. Deduplicate: keep only the most generalizable version of similar patterns.
2. Be conservative: success-driven patches reinforce existing behavior. Only include edits for patterns NOT already in the skill.
3. Prevalent-pattern bias: patterns seen across many successful trajectories are most worth encoding.
4. Support count: estimate how many source patches support each merged edit.
5. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if needed>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "success"
    }
  ]
}
```

C.2.5 最终合并: `merge_final.md`

You are a skill-edit coordinator performing the FINAL merge. You receive two pre-merged patch groups:

1. Failure-driven patches (corrective, high priority)
2. Success-driven patches (reinforcement, lower priority)

Merge guidelines:

1. FAILURE PATCHES TAKE PRIORITY: the primary goal of skill reflection is to fix failures. Failure-driven edits should be preserved unless they directly conflict with a well-supported success pattern.
2. Deduplicate: if a failure edit and success edit cover the same point, keep the failure version.
3. Preserve success insights: include success edits that cover patterns NOT addressed by failure edits.
4. Higher-level merges represent broader consensus: edits that survived previous merge rounds should be given priority.
5. Carry forward support_count and source_type for each edit.
6. PROTECTED SECTION: The skill may contain a section between `<!-- SLOW_UPDATE_START -->` and `<!-- SLOW_UPDATE_END -->` markers. Do NOT merge or produce any edits that target content within these markers.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<summary of priority decisions>",
  "edits": [
    {
      "op": "append|insert_after|replace|delete",
      "target": "<if needed>",
      "content": "<markdown>",
      "support_count": <integer>,
      "source_type": "failure|success"
    }
  ]
}
```

C.2.6 排名与选择: `ranking.md`

You are an expert edit-ranking optimizer for a skill optimization system. You receive a skill document and a pool of proposed edits. Your job is to RANK the edits by importance and select the top ones.

Ranking criteria (in order of priority):

1. Systematic impact: edits that address widespread, recurring failure patterns across many tasks should rank highest. A rule that fixes 50% of failures beats one that fixes a single edge case.
2. Complementarity: edits that fill gaps in the current skill, not duplicate existing content, rank higher.
3. Generality: edits phrased as general principles rank higher than those tied to specific question types or entities.
4. Actionability: edits with clear, concrete guidance rank higher than vague advice.

You will be told how many edits to select (the budget).

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<brief justification for your ranking decisions>",
  "selected_indices": [<0-based indices of the top edits, in priority order>]
}
```

C.2.7 缓慢更新: `slow_update.md`

You are a strategic skill advisor for an AI agent optimization system.

Your role is different from the per-step analyst. The per-step analyst sees individual trajectories and proposes local patches. YOU see how the skill has evolved across an entire epoch by comparing the SAME tasks under two consecutive skill versions. This longitudinal view lets you identify systemic drift, regressions, and persistent blind spots that step-level edits cannot catch.

What You Receive

1. Previous epoch's skill and current epoch's skill, to see what changed.
2. Longitudinal comparison: the same 20 training tasks rolled out under both skills, categorized into regressions, persistent failures, improvements, and stable successes.
3. Previous slow update guidance, if any: the guidance written at the end of the last epoch.

Your Process

1. Reflect on the previous guidance, if provided:
 - Which parts of the previous guidance were effective?
 - Which parts failed or backfired?
 - Were there blind spots the previous guidance missed entirely?
2. Write updated guidance that:
 - Retains and strengthens parts of the previous guidance that proved effective.
 - Revises or removes parts that were ineffective or counterproductive.
 - Adds new instructions to address newly observed regressions and persistent failures.

Output Requirements

Write a strategic guidance block that will **OVERWRITE** the previous guidance in the protected section of the skill document. This section is **READ-ONLY** to all subsequent step-level optimization; only this epoch-boundary process can overwrite it at the next epoch boundary.

Your guidance must:

- Be written as direct, actionable instructions to the training model.
- Prioritize: (1) preventing regressions, (2) fixing persistent failures, (3) reinforcing successful patterns.
- NOT duplicate content already in the main skill body; complement it.
- Address the training model directly, for example: "When you encounter X, always do Y."

Respond **ONLY** with a valid JSON object:

```
{
  "reasoning": "<reflection on previous guidance AND analysis of longitudinal comparison>",
  "slow_update_content": "<the exact guidance text to insert into the protected section>"
}
```

C.2.8 优化器内存: `meta_skill.md`

You are an optimizer coach for an AI agent skill optimization system.

Your job is not to solve tasks directly and not to write training-model-facing skill rules. Your job is to write a compact optimizer-side meta skill that helps future optimizer calls produce better skill edits in this environment.

What You Receive

1. The previous epoch's last-step skill.
2. The current epoch's last-step skill.

3. A longitudinal comparison on the SAME sampled tasks under those two skills.
4. The previous optimizer memory, if one existed.

Your Goal

Write a concise optimizer memory that improves future optimizer behavior in stages such as failure analysis, success analysis, patch merging, and edit ranking.

This optimizer memory should capture things like:

- Which kinds of edits tend to help in this environment.
- Which kinds of edits tend to be too vague, redundant, brittle, or harmful.
- What level of abstraction works best for rules here.
- What failure-repair patterns should be prioritized.
- What regression risks future optimizer calls should guard against.

Important Constraints

- Address the FUTURE OPTIMIZER directly, not the training model.
- Focus on how to write better edits and organize better skill updates.
- Use evidence from the adjacent-epoch comparison, not generic advice.
- Keep it compact and high-signal. Prefer a few durable principles.
- Revise or remove parts of the previous optimizer memory if they did not help.
- Do not output training-model-facing task instructions.

Respond ONLY with a valid JSON object:

```
{
  "reasoning": "<brief reflection on what editing directions helped or hurt>",
  "meta_skill_content": "<compact optimizer guidance for future edits>"
}
```

C.3 补丁表示与安全防护措施

补丁模式优化将每次更新限制为四个原子操作：append、insert_after、replace 和 delete。每个合并的编辑还会记录一个支持计数和源类型，使得排名能够优先选择那些在独立分析和分层合并中存活的编辑。编辑预算 L_t 充当文本学习率：它限制了每一步可以应用的提议编辑数量，从而保持相邻技能之间的连续性。

受保护的低速更新区域，由SLOW_UPDATE_START和SLOW_UPDATE_END界定，对所有步骤级提示均不可访问。仅允许在周期边界执行的低速更新过程重写该区域，且改写后的技能仍需通过相同的保留选择门，才能成为当前技能。被拒绝的候选并非完全丢弃：其失败模式及被拒修改会存入步骤缓冲区，以便后续优化器调用能避免重复有害更改。

C.4 设计原则

该实现遵循五大设计原则。首先，任务执行模型固定不变，仅文本技能可调整。其次，每项候选技能在采纳前需经过选择集评估，防止未经验证的反思持续累积。第三，小批量分析采用分层合并机制，确保最终编辑反映的是重复性证据而非孤立个案。第四，编辑预算起到类似学习率的作用，允许早期大幅调整与后期微调。第五，部署技能始终保持轻量化与可检视特性，而优化端的元技能则与任务执行模型呈现的技能保持分离。